

コードの時系列変化を考慮した保守性低下 の要因分析と改善

鈴木研究室 笹川 尋翔

1.1 開発現場の課題

持続的なソフトウェア開発が困難

76%の開発者がリファクタリングによるバグ混入やリグレッションのリスクを感じている [1]

ソフトウェアが時間とともに複雑化 -> 開発者の認知負荷が増加

1.2 リファクタリング環境の変化

CI/CDによる継続的な品質評価の普及

利点: テストの自動化、カバレッジやメトリクスを追跡が可能に

欠点: 単一時点でのコード分析に留まっており、変化を追跡できない

クラスや関数の状態の変化から、より有益な情報を得られるのでは？

2 関連研究

- レビューコメントによるコードスメルの分析^[2]
 - レビューアーは保守性の低下要因をコーディング規約から特定
- 静的解析ツールによるリファクタリング支援^[3]
 - ツールによりリファクタリングの機会が増えるが、問題の背後にある理由を理解するのが難しい

[2] X.Han et al., "Understanding Code Smell Detection via Code Review: A Study of the OpenStack Community"

[3] S.Romano et al., "Do Static Analysis Tools Affect Software Quality when Using Test-driven Development?"

3 目的

- 時系列変化量を用いたバグ予測精度の向上
 - 保守性メトリクス（例: コード行数、循環的複雑度）の変化量を導入・分析
- バグの特徴と保守性メトリクスの関係性の解明
 - バグの特徴を体系的に整理し、保守性とのつながりを明確化
 - バグの性質ごとにコードの改善策を示す

4.1 時系列変化量の追跡と利点

従来の分析手法の限界

従来の静的解析は「スナップショット分析」

- 経験則による評価 -> プロジェクトの性質が無視される

変化に注目する

「コードがどう変わったか？」を追跡

- 前回の複雑度が10、今回は15 -> 複雑度が5増加

4.2 比較対象データの構築

スナップショット分析のためのデータとして、BugHunter Dataset^[4]を使用

このデータセットに保守性メトリクスの変化量を追加

追加前と追加後で、評価指標の有意な改善が見られるかを検証

[4] R.Ferenc et al., "An automatically created novel bug dataset and its validation in bug prediction"

4.3 ランダムフォレストによるバグ予測

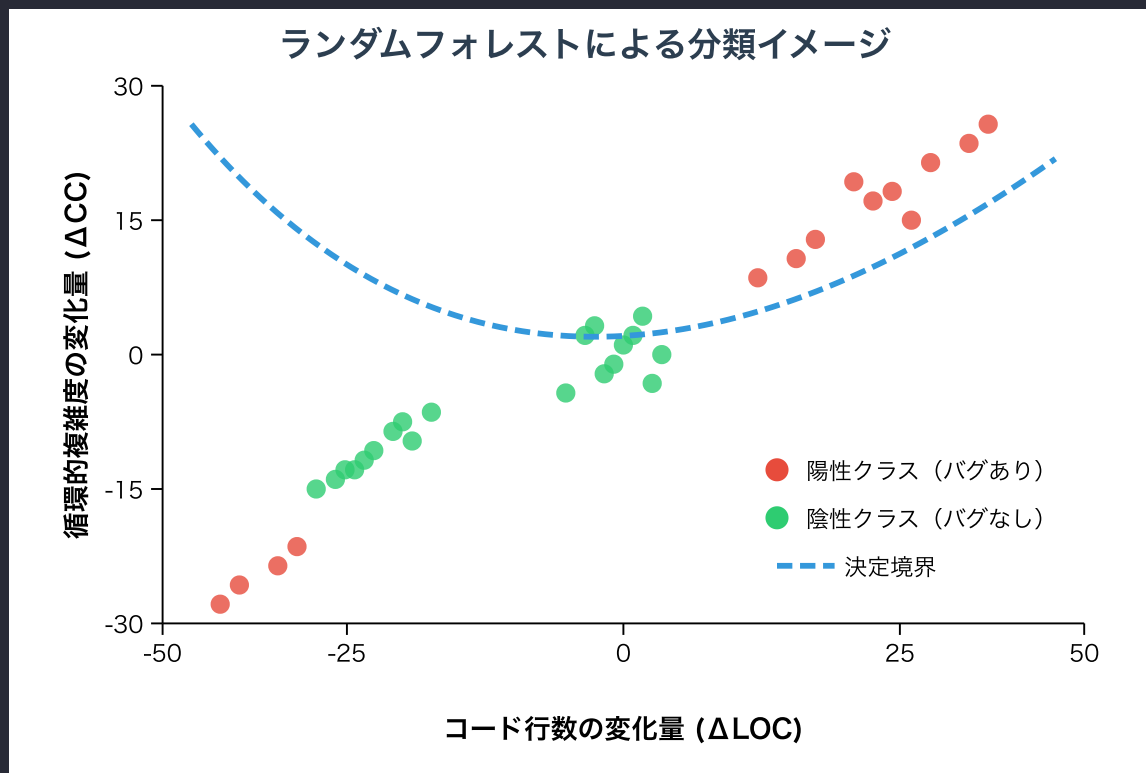


図1 分類イメージ

複数の決定木を組み合わせた機械学習アルゴリズム

- 各決定木が独立して予測を実行
- 多数決で最終的な分類結果を決定

5.1 プロジェクトの選定

表1 選定したプロジェクト

プロジェクト	役割
antlr4	構文解析ツール
BroadleafCommerce	Webアプリフレームワーク
ceylon-ide-eclipse	IDEプラグイン
elasticsearch	全文検索エンジン
hazelcast	インメモリデータベース
oryx	機械学習フレームワーク

5.2 プロジェクトの分析

分析手法

- 特徴量重要度（Feature Importance）の測定
- ヒストグラムによる特徴量分布の確認
- Partial Dependence Plot（PDP）による分類傾向の把握
- 決定木の可視化による分類条件の可視化

6.1 評価指標の測定

- 3プロジェクト: F1スコアが最大0.1向上（有意差あり）
- 2プロジェクト: F1スコアが向上（有意差なし）

F1スコアの伸びに伴い、Precision、Recall、Accuracyも向上

全体的に、評価指標の値が改善される傾向が見られた

6.2 特徴量重要度

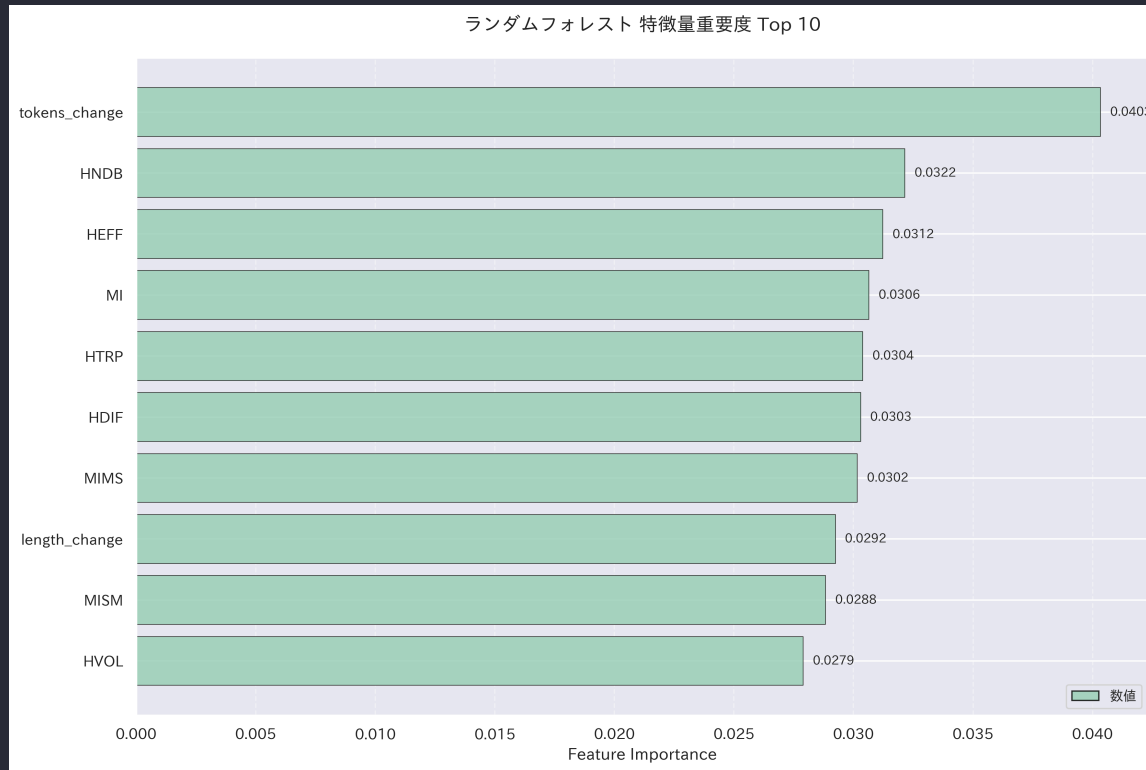


図1 hazelcastの特徴量重要度

- トークン数・コード行数の変化量の重要度が高い
- Halsteadメトリクス、Maintainability Index (MI) も上位に位置している

6.3 特徴量分布

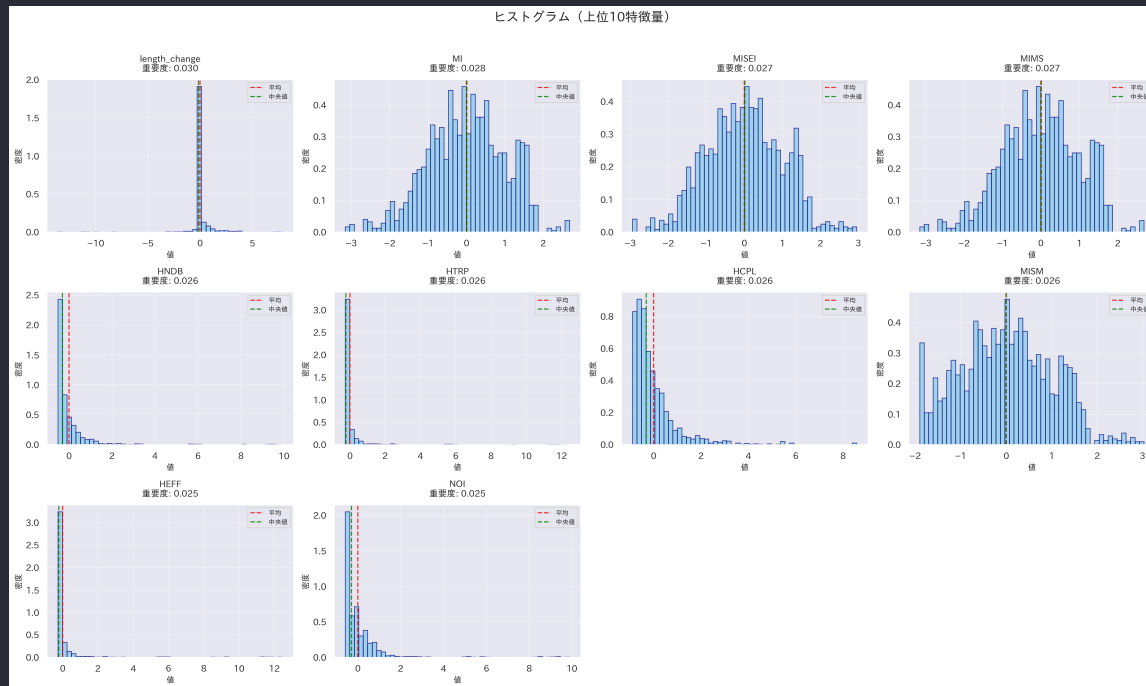


図2 ceylon-ide-eclipseの特徴量分布

- 変化量・Halsteadメトリクスは分散が小さく、特定の箇所に値が集中
- MIは分散が大きい
- 一峰性の分布が多い

6.4 PDP分析



図3 elasticsearchのPDP

- トークン数・コード行数の変化量が0付近のとき: 陽性クラスの予測確率が上昇
- ほとんどの特徴量において、陽性クラスの予測確率が0.5未満

6.5 決定木分析

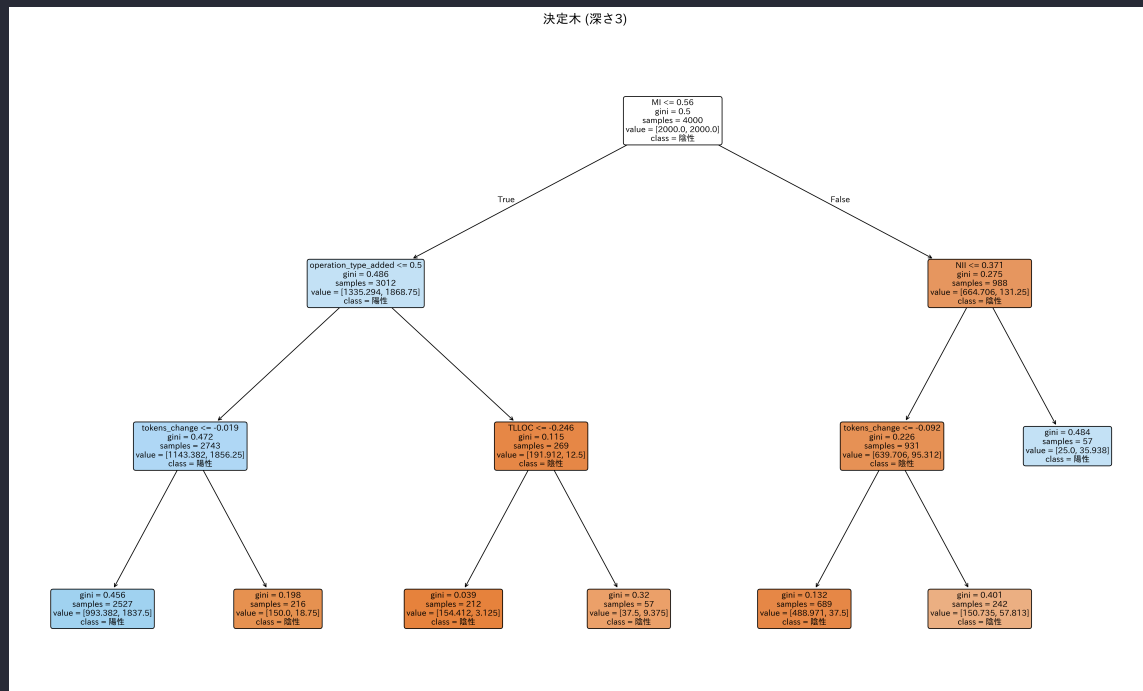


図4 elasticsearchの決定木

- 陰性クラスのノードのジニ不純度が比較的低い
- 高い確信度で陰性クラスを分類している

7 評価・考察

陰性クラス予測の改善

リファクタリングによるメトリクスの減少傾向を確認できた

消去法的な分類がF1スコアの改善に寄与

さらなる精度向上に向けて

PDPや決定木を見ると、陽性クラスの予測確率が低い

バグ混入理由が多様であることが影響している？

8 今後の見通し

陽性クラスの詳細な分類による因果関係の解明

バグ混入と保守性メトリクスに関連性が不明確であるため

バグの体系的分類によるパターンの識別

Orthogonal Defect Classification (ODC) のような分析手法を活用

バグの特徴とコードメトリクスの時系列変化の関連性を示したい

9 まとめ

6つのプロジェクトのうち3つで有意差が認められ、F1スコアが最大0.1向上

保守性メトリクスの変化量を組み合わせることで、より効果的なバグ分類ができるようになった

実用性向上に向けて

陽性クラスの分類精度を高め、品質改善に役立つコードの特徴を捉えたい

質疑応答用資料

保守性とは

- JIS X 0129による定義
 - 修正のしやすさに関するソフトウェア製品的能力
 - 修正: 「是正」、「向上」、「環境の変化、要求仕様や機能仕様の変更」にソフトウェアを適応させること」
- 保守性メトリクスの例
 - 複雑度: 循環的複雑度、ネストの深さ
 - 結合度: ファンイン/ファンアウト
 - 凝集度: ある変数を参照しているメソッドの割合
 - サイズ: コード行数、メソッド数

BugHunter Datasetについて

- 特定のファイル、クラス、メソッドのデータセット
- バグ混入コミット（陽性）とバグ修正コミット（陰性）を含む
- コミットごとのコードメトリクスを計算・記録
- コミットIDと識別子を使用してデータセットを拡張できる